Martin Gonzalez Rico

Modificadores, static e inicialización

No.	Respuesta	Explicación:
1	d.Clase Padre Clase Hija	Esta es la respuesta porque cuando creamos un objeto de la clase Hija, primero se ejecuta el constructor de Padre. Esto pasa porque Java llama automáticamente a super() si no lo escribimos. Después de terminar el constructor del padre, se ejecuta el constructor de la hija, por eso primero imprime Clase Padre y después Clase Hija.
2	a.Dog dog = new Dog(); b.Animal dog = new Dog();	Las dos opciones crean un objeto con new Dog(). En la opción a, la referencia es del tipo Dog, que coincide con el objeto creado. En la opción b, la referencia es del tipo Animal, pero también es válida porque Dog hereda de Animal. En cambio, Dog dog = new Animal() no funciona porque un objeto padre no se puede asignar directamente a una referencia de la hija.
3	b.sc = asc;	AnotherSampleClass hereda de SampleClass. Por eso, un objeto de la clase hija se puede guardar en una referencia de la clase padre sin necesitar un cast.
4	c.Child	Aunque la variable obj está declarada como Parent, el objeto que se crea realmente es new Child(). Como Child sobrescribe el método display(), Java ejecuta la versión correspondiente al objeto real durante la ejecución. Por eso imprime Child y no Parent.
5	d.Bark	La variable myDog tiene el tipo declarado Animal, pero apunta a un objeto Dog. Como Dog sobrescribe makeSound(), se ejecuta su implementación y se imprime Bark.
6	b.Ladrido.	miAnimal apunta a un objeto de tipo Perro, y la clase Perro sobrescribe el método sonido() de Animal. Cuando se llama al método, Java utiliza la implementación de Perro, por lo que imprime Ladrido..
7	a.Child	obj ya apunta a un objeto Child, así que convertir la referencia a Child mediante (Child) obj es válido. Como Child sobrescribe display(), se ejecuta esa versión y se imprime Child.
8	b.sc: class SampleClass asc: class AnotherSampleClass	El método getClass() devuelve la clase real del objeto en tiempo de ejecución. En este caso, sc apunta a un objeto creado con new SampleClass(), mientras que asc apunta a uno creado con new AnotherSampleClass(). Por eso cada variable muestra la clase correspondiente al objeto que tiene asignado.
9	d.sc: class AnotherSampleClass asc: class AnotherSampleClass	Al ejecutar sc = asc, ambas variables apuntan al mismo objeto de tipo AnotherSampleClass. Aunque sc esté declarada como

		SampleClass, eso no cambia la clase real del objeto. Por eso las dos llamadas a getClass() devuelven AnotherSampleClass.
10	c.Hash is: 111111, 44444444, 999999999	SampleClassA y SampleClassB sobrescriben getHash(). Aunque las referencias scA y scB son de tipo SampleClass, los objetos reales son de las clases hijas. Por eso Java ejecuta la implementación correspondiente a cada objeto: 111111 para SampleClass, 44444444 para SampleClassA y 999999999 para SampleClassB.
11	a.A's constructor B's constructor C's constructor	Cuando creamos un objeto de la clase C, Java debe ejecutar primero los constructores de sus clases padre. C hereda de B y B hereda de A, así que primero se ejecuta el constructor de A, luego el de B y finalmente el de C.
12	b.It will print "Base constructor with message: Hello" followed by "Derived constructor"	La respuesta es correcta porque Derived llama a super("Hello"), lo que selecciona el constructor de Base que recibe un String. Ese constructor imprime primero Base constructor with message: Hello. Cuando termina, continúa el constructor de Derived e imprime Derived constructor.
13	d.Compilation fails.	El código no compila porque Gamma es una subclase de Beta, y Beta a su vez hereda de Alpha. Se puede asignar un objeto de una subclase a una referencia de una superclase, pero no al contrario de forma automática. Por eso no podemos guardar directamente un objeto Alpha o Beta en una referencia Gamma.
14	c.Compilation error	El método show() de Parent es protected, pero en Child se declara sin modificador, lo que significa que tiene acceso default. Eso reduce la visibilidad del método original y Java no lo permite al sobrescribir. Para que compile, el método de Child tendría que ser protected o public.
15	a.message middle end	Primero new CA("message ") llama al constructor que recibe un String e imprime message . Después, al crear new CB(), Java utiliza el constructor por defecto de CB, que llama automáticamente a super() y ejecuta el constructor sin argumentos de CA, imprimiendo middle. Finalmente se imprime end. Por eso el resultado completo es message middle end.
16	b.void foo() { /* more code here */ } c.public void foo() { /* more code here */ } e.protected void foo() { /* more code here */ }	La clase One tiene el método void foo() sin modificador de acceso, por lo que es accesible dentro del paquete. Al sobrescribirlo en Two, podemos mantener ese mismo nivel de acceso o ampliarlo a protected o public. No podemos cambiar

		el retorno de void a int manteniendo la misma firma, ni reducir la visibilidad a private.
17	c.void display() { System.out.println("InChild"); } d.public void display() { System.out.println("Child"); }	La opción c es válida porque mantiene el mismo nombre, los mismos parámetros y el mismo acceso de paquete que el método original. La opción d también es válida porque utiliza la misma firma y amplía la visibilidad a public. La opción a no funciona porque intenta convertir un método de instancia en static, y la opción b reduce la visibilidad a private.
18	d.public char met() { return '2'; }	La clase padre declara met() sin parámetros y devuelve un String. En la clase hija, declarar otro método con la misma firma significa sobrescribirlo, así que el tipo de retorno debe ser compatible con el de la clase padre. char no es compatible con String, por lo que esa opción no compila. Las opciones que reciben un parámetro sí pueden ser sobrecargas distintas.
19	a.La herencia representa una relación "es-un" donde una clase deriva de otra clase. b.La composición representa una relación "tiene-un" donde una clase contiene una instancia de otra clase. d.Tanto la herencia como la composición pueden usarse juntas para modelar relaciones complejas. e.El uso (o asociación) representa una relación "usa-un" donde una clase utiliza métodos o instancias de otra clase. f.Se debe preferir la composición sobre la herencia para fomentar la reutilización de código y la flexibilidad.	La herencia representa una relación de «es-un», como Perro es un Animal. La composición representa una relación de «tiene-un», donde una clase contiene o utiliza un objeto de otra clase. La asociación también puede representar una relación de «usa-un». Estas relaciones pueden combinarse para modelar sistemas más complejos, y normalmente se recomienda preferir la composición sobre la herencia cuando se busca reducir el acoplamiento y mejorar la flexibilidad.
20	d.The Dog barks	animal apunta a un objeto Dog y Dog sobrescribe el método speak() de Animal. Aunque la variable esté declarada como Animal, Java ejecuta la versión del objeto real, que imprime The Dog barks.
21	a.Compilation fails.	El código no compila porque el método speak() de Animal es public, pero en Dog se intenta sobrescribir como protected. Java permite mantener o ampliar la visibilidad de un método al sobrescribirlo, pero no reducirla.
22	c.Java Jeve va	Cada objeto ejecuta su propia versión sobrescrita de mensaje(). Alpha devuelve el texto sin cambios, Beta reemplaza las letras a por e y devuelve Jeve, y Gamma utiliza substring(2), que devuelve el texto desde el índice 2, es decir, va.

23	b.Sound Roar Bark Growl	Primero animal.makeSound() llama al método de Animal e imprime Sound, y después animal.makeSound("Roar") imprime Roar. La variable dog está declarada como Animal, pero apunta a un objeto Dog, así que dog.makeSound() ejecuta la versión sobrescrita e imprime Bark. Por último, dog.makeSound("Growl") utiliza el método que recibe un String, heredado de Animal, y por eso imprime Growl.
24	c.Feline cougar c c	Primero se ejecuta el constructor de Feline y se imprime feline; después se ejecuta el constructor de Cougar y se imprime cougar. En go(), el atributo heredado type cambia de "f" a "c". Como Cougar no declara otro atributo type, tanto this.type como super.type hacen referencia al mismo campo heredado. Por eso al final se imprime cc.
25	a. ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: ElectricCar	Cada clase declara su propio atributo type, ocultando el de la clase padre. Cuando hacemos ElectricCar myCar = new ElectricCar() y accedemos a myCar.type, se utiliza el atributo de ElectricCar, que contiene "ElectricCar". A diferencia de los métodos sobrescritos, los atributos no se resuelven mediante el objeto real, sino según el tipo de referencia.
26	d.Compilation fails because of an error in line 18.	El constructor de Employee no llama explícitamente a super(...), así que Java intenta insertar automáticamente una llamada a super(). El problema es que Person solamente tiene un constructor que recibe un String y no tiene constructor sin argumentos. Por eso no se puede completar la llamada al constructor padre y el código no compila.
27	f.Atom granite atom granite.	Cuando creamos new Mountain(), el constructor llama a super("granite "), por lo que primero se ejecuta el constructor de Atom, que imprime atom, y luego el de Rock, que imprime granite. Después se crea otro objeto con new Rock("granite "), lo que vuelve a ejecutar el constructor de Atom y después el de Rock. Por eso los mensajes aparecen dos veces: atom granite atom granite.
28	a.1234	La creación new Y(5) llama primero al constructor Y(int), que delega en Y() mediante this(). Después Y() llama a X(int) mediante super(6), y este último llama a X() utilizando this(). Los constructores más internos terminan primero y después continúa la ejecución de los demás; por eso se imprimen 1, 2, 3 y 4.
29	d.Square square = new Square (); square.foo (); square.foo("bar");	La primera instrucción crea un objeto Square y ejecuta el constructor de Shape, que imprime Shape: constructor.

		Después square.foo() llama al método que utiliza super.foo(), por lo que imprime Shape: foo. Finalmente, square.foo("bar") utiliza la sobrecarga que recibe un String e imprime Square: foo. Así se obtiene el orden solicitado.
30	d.d	En esta opción se crea un objeto con new A() y después se intenta convertir a B mediante (B) ob. El problema es que el objeto real es de tipo A, no de tipo B. Aunque B herede de A, un objeto padre no se convierte automáticamente en un objeto hijo. El cast compila, pero falla en tiempo de ejecución y produce ClassCastException.

31	b.p2 = (ClassC)p4;
	d.p1 = (ClassB)p3;
	e.p0 = p1;
		La opción b, p2 = (ClassC)p4, es válida porque p4 apunta realmente a un objeto ClassC; el cast confirma ese tipo. La opción d, p1 = (ClassB)p3, también es válida porque p3 apunta a un objeto ClassB. Finalmente, p0 = p1 funciona sin cast porque una referencia ClassA puede apuntar a un objeto ClassB, ya que ClassB hereda de ClassA.

32	c.If line 12 is removed, the compilation succeeds.	La línea String str1 = (String) build1; es el problema porque build1 apunta a un objeto Building y String es una clase que no tiene una relación de herencia compatible con Building. El cast no puede hacer que estos tipos no relacionados sean compatibles. Las otras asignaciones pueden compilar, aunque el cast de Building a Barn podría fallar en tiempo de ejecución si se ejecutara con el objeto creado en el código.
33	d.The code runs with no output.	La variable g apunta a un objeto Epsilon, que hereda de Gamma e implementa la interfaz Omega. Por eso las conversiones a Omega y Epsilon son válidas. Finalmente, m = e también es válido porque un objeto Epsilon puede guardarse en una referencia de tipo Omega.
34	b.true	Las tres condiciones son verdaderas. v es un String, así que v instanceof String devuelve true. sb es un StringBuffer, que también hereda de Object, así que sb instanceof Object devuelve true. Por último, MyList hereda de ArrayList, por lo que new MyList() instanceof ArrayList también devuelve true. Como las tres condiciones son verdaderas, el operador && devuelve true.
35	g.Compilation fails	El código no compila porque la clase Foo no declara ningún atributo llamado a, aunque Bar sí lo tenga.
36	b.Change line 13 to: public Sub() { super(5); } d.Change line 13 to: public Sub() { this(5); }	El constructor Sub() tiene dos problemas: intenta acceder al atributo a, que es private en Super, y no llama a un constructor válido de la clase padre sin argumentos. La opción b soluciona

		esto con super(5), que llama directamente al constructor existente de Super. La opción d utiliza this(5), que llama al constructor Sub(int), y ese constructor ya llama correctamente a super(a)
37	a.protected long blipvert(long x) { return 0; } b.public int blipvert(int x) { return 0; } c.private int blipvert(long x) { return 0; } d.protected long blipvert(int x, int y) { return 0; } f.protected int blipvert(long x) { return 0; }	las opciones a, c y f cambian el parámetro de int a long, así que son sobrecargas y pueden tener distintos modificadores de acceso. La opción d también es válida porque utiliza dos parámetros, por lo que es otra sobrecarga. La opción b sobrescribe correctamente el método original ampliando su visibilidad de protected a public. No funcionan la opción e, porque cambia el tipo de retorno de int a long al sobrescribir con la misma firma, ni la g, porque reduce el acceso a private.
38	c.Line 19 d.Line 20 f.Line 22	La línea 19 funciona porque convierte speakIT a Truth antes de llamar al método; el objeto real es un Tell, que implementa esa interfaz. La línea 20 también funciona porque tellIt ya está declarado como Tell, que tiene el método tellItLikeItIs(). La línea 22 es válida por el mismo motivo que la 19: primero hace el cast a Truth y después llama al método. Las líneas 18 y 21 no funcionan porque intentan convertir el resultado de una llamada void, y la línea 17 tampoco es válida porque Speak no declara ese método.
39	c.Valor de la clase C 28	Aunque la referencia ac está declarada como A, apunta a un objeto C. Por eso, cuando llamamos a ac.ejemplo(), se ejecuta la versión sobrescrita de C, que devuelve "Valor de la clase C". Sin embargo, los atributos no funcionan como los métodos sobrescritos: ac.x accede al campo x de A, que sigue valiendo 8. Por eso ac.x + 20 da 28.
40	a.Animals make noise	Los métodos private no se heredan ni se sobrescriben. Aunque Dog declare otro método llamado speak(), ese método es independiente del método privado de Animal. La llamada animal.speak() se realiza desde el propio código de Animal, donde el método privado es accesible, así que se ejecuta la versión que imprime Animals make noise.